Generative adversarial networks in generative chemistry

Jakub Czerwonka

1 Abstract

Due to significant improvements over the last decade, deep learning[2] is widely used across many fields of science. One application of deep learning is in drug discovery. Drug discovery is the field of pharmacology in which we try to find sequences of chemical molecules that can be used as drugs.

In this work we want to introduce model of modified transformer[17] and modified generative adversarial network[3], that try to generate sequence of molecules, which desirable properties. This kind of Generative adversarial network, named TGSGAN (Transformer Gumbel Softmax GAN) is based on Gumbel-Softmax method[12][4], which solves problem of weak gradients in generator-discriminator structure, while training of sequences. Model of modified transformer and GAN is going to be evaluated on qm9 database [16], where we will give test set properties to a model, generate sequence besed of those properties, and we will calculate the model's error between ground truth and properties of generated sequences. All the code is on the given repository (https://github.com/JakubCzerwonka/TGSGAN).

2 Introduction

2.1 Generation sequence of molecules

Generation of molecular sequence with desirable properties with deep learning is widely researched over the last decade. This can lead to find many of kinds of useful molecules for drug discovery. One of the more popular works regarding generation of sequence of molecules is the use of Junction Tree Variational Autoencoder [5], where researchers used variational Autoencoder and MolGAN [1], where has been used generative adversarial network to generate sequence of molecules, based on some chemical properties given by the user.

2.2 Why GANs?

Generative adversarial networks are among the most intriguing generative models, due to a specific type of training that resembles a game in which two al-

gorithms compete with each other, improving simultaneously. Generative adversarial networks trained this way exhibit a more "free" exploration of features, leading to the generation of more diverse and interesting molecules. Sequence models trained using the classical maximum-likelihood method often lack a mechanism to incorporate more diverse molecules into the model itself, which is not typically what we want in drug discovery. Despite the lack of an in-built method to generate diverse molecules in typical implementations, there are methods to increase diversity that come with disadvantages. GANs, however, despite lacking an element to generate more diverse samples, still place strong emphasis on diversity in generated samples. If a generator, despite the correctness of its generated samples, continues to generate the same samples, the discriminator, due to its training, will easily distinguish generated samples from ground truths, forcing the generator to generate different, more diverse samples.

2.3 Generative adversarial network

The classical model of a generative adversarial network consists of a generator and a discriminator, both typically implemented as artificial neural networks. Generator aims to generate the most similar probabilities based on training data $\mathcal{X}$, when discriminator has to distinguish which data is from training set or generated by generator. In other words, the generator is trying to generate the most similar samples to the training data to fool the discriminator, which has to recognize which are ground truth and which are fake (generated). It is similar to the process when counterfeiters try to make the most similar money, while police try to distinguish which is fake.

To train distribution of generator p_{gen} on training set $x \in \mathcal{X}$ we define $p_z(z)$, which is white noise, and then we put this to differentiable mapping $G(z; \theta_{gen})$, where θ_{gen} are parameters of generator and G is generator. Let us also define another mapping $D(x, \theta_{disc})$, where D is the discriminator and, and θ_{disc} are its parameters. Both the generator and discriminator, while training, play the game below

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[logD(x)] + \mathbb{E}_{z\sim p_z(z)}[log(1-D(G(z)))], \quad (1)$$

where $z \sim \mathcal{N}(0,1)$, which is gaussian noise.

In this classical approach to training a generative adversarial network, we use a min-max game, where maximum likelihood estimation is not typically used in training sequence models. Training with classical maximum-likelihood methods usually yields better results, but unfortunately, it is less diverse.

3 Machine learning in drug discovery

Machine learning has gained significant interest over the last decade due to significant improvements in this field. In many studies, deep learning is used

to solve problems, improve tasks, and support other profit activities, such as drug discovery. For example, a major accomplishment was the development of a model by DeepMind[6] to estimate the 3D structures of molecules. Another example is a regression model that predicts numerical properties of given molecules[9]. Drug discovery is the process of identifying molecules that can be useful in medical treatment. Generative algorithms have been examined for their ability to generate potentially useful molecules, demonstrating the utility of those models. For example, in the paper "Junction Tree Variational Autoencoder for Molecular Graph Generation"[5],an autoencoder is used, and in "MolGAN: An implicit generative model for small molecular graphs"[1], a generative adversarial network is used. Machine learning in drug discovery can be used in many ways; for example, models that extend sequences have proven useful in medical treatment. Some models also aim to generate useful sequences of molecules from scratch based on given properties, a process called "de novo drug design". The use of machine learning in drug design has many advantages over more classical methods, for example, lower cost. Despite the advantages, models also encounter some problems. One disadvantage is that the model's predictions contain some errors. Models that predict numerical properties of chemical compounds often exhibit small numerical errors, which can exceed the norm. Another problem is that the generated parts of sequences are not chemically correct (assuming, for example, valence-bond theory). A major disadvantage is also the molecule's unknown or even impossible synthesizability due to chemical reactions.

3.1 Methods of representation of the structures of molecules

To use machine learning models, we have to frame the problem as an optimization problem and potentially change how the data is constructed. In sequence generation, molecules are usually represented as graphs or strings, such as SMILES or SELFIES[9]. SMILES (Simplified Molecular Input Line Entry Specification) is a method to represent molecules as a simple set of atoms and supporting elements. For example, benzene is "C1=CC=CC=CC1", where "C" represents a carbon atom, "1" marks the beginning and end of the chemical ring, and "=" represents a double bond. SMILES coding, when directly tokenized as numbers, usually does not work well because SMILES is not robust to small sequence errors. For example, if a generative model generates a SMILES sequence without ring closure, we do not really know exactly which sequence the model intended to generate. Therefore, researchers introduced an alternative representation of molecules in sequences, named SELFIES. SELFIES are robust to small sequence errors, making molecular generation easier. Following benzene written by SELFIES looks like: [C][=C][C][=C][C][=C][Ring1][=Branch1], while [C] represents carbon atom, [=C] represents a carbon atom with double bonding to the previous atom or functional group, [Ring1] represents the closure of the ring, and [=Branch1] is a supporting element in ring notation. Compared SMILES benzene (C1=CC=CC=CC1) and SELFIES benzene ([C][=C][C][=C][C][=C][Ring1][=Branch1]), there is no need to extend sequence

integrity at the beginning and end of the sequence, freeing the model from unnecessary work and improving its performance.

3.2 Data processing for the model

To use a generative adversarial network based on an artificial neural network, we usually must encode data into a numerical representation rather than a string representation. There are many ways to encode data into a numerical representation, such as graph representations or molecular fingerprints, which describe molecules as vector representations with usually only binary variables. An example of a molecular fingerprint is [0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0], corresponding to benzene, where the vector length is a hyperparameter. However, tokenization is usually a much better method because of its ease. Tokenization is the process of assigning a number to individual letters or words in sequences. SELFIES benzene after example tokenization looks like: [18, 9, 18, 9, 18, 9, 27, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], where "18" represents "[C]", "9" represents "[=C]", "27" represents "[Ring1]", "7" represents [=Branch1], and "0" represents a padding token, which ensures a fixed size of the vector, usually needed in artificial neural networks.

4 Architecture

4.1 Combining sequences of molecules with properties

There are many methods for generating correct molecules with given properties. One of the methods is to train an autoencoder, which takes as input a sequence of molecules and the properties of that molecule to create, by the encoder, a continuous representation, named latent representations, that can be put to another algorithm, which would generate, depending on that latent representation, correct sequences and properties. The problem with that approach is that it has disadvantages, such as incorrect training. Usually, when we train an autoencoder on complex tasks, we do not know whether the latent representation is well-defined. Let us assume we have a well-trained autoencoder on a database with only drawings of numbers. If we show representations of images for two dimensions, there would not be a real problem in showing that the latent representations of the numbers are correct, because all the same numbers would be close to each other. However, there would be a problem with complete certainty if the samples are correctly grouped. Of course, we can see the use of similar atoms, but in chemistry, even molecules with the same atoms can be very different due to their arrangement. There are some metrics to ensure the similarity of the molecules, like Tanimoto similarity, but this approach is still not fully robust to errors. Also, because properties usually have less dimensionality than sequences of molecules, it is quite obvious that the model, instead of paying equal attention to sequences and properties, would focus mostly on properties, which is not quite what we want either. Instead of using autoencoders, we can

use modified generative adversarial networks without direct latent representations, as in MolGAN, which used reinforcement learning to generate molecules with expected properties.

4.2 Generator

The generator in TGSGAN takes fixed-size property-value vectors and a tokenized SELFIES sequence. The generator is a modified transformer without an encoder. In stead of a classical encoder, which takes sequence y_t, there is a projection layer, which takes the properties vector $v^{(i)}$, which contains a fully connected layer with $T \cdot d_{model}$ neurons, to ensure correct size, passed to the decoder, while the decoder takes y_t (Note: the decoder does not take the time-shifted sequence). Such a projection not only ensures a correct size compatible with cross-attention but also serves as a kind of "memory" that lets the model know which sequence has which properties. In other words, let's define $y_t^{(i)}$ as input sequence , such that $y_t^{(i)} \in \mathcal{X}$ and vector $v^{(i)}$, such that $v^{(i)} \in \mathcal{X}$ and $v^{(i)} \in \mathbb{R}^m$, where m is constant, what is number of molecules' properties. While training generator G we pass vectors with fixed size $v^{(i)}$, where there is projection $h^{(i)} = proj_{dense}(v^{(i)}, W, b) = Wx + b$, where the weights $W \in \mathbb{R}^{n \times T \cdot d_{model}}$, bias term $b \in \mathbb{R}^{T \cdot d_{model}}$, T is sequence length and d_{model} it is hyperparameter of the transformer. Vector $h^{(i)}$, such that $h^{(i)} \in \mathbb{R}^{T \cdot d_{model}}$ is passed to transformer's decoder getting sequence, such that $\hat{y}_t^{(i)} = G(v^{(i)}; \theta_{gen})$, where $\hat{y}_t^{(i)} \in \mathbb{R}^{T \times \mathfrak{v}}$, where $\mathfrak{v}$ is constant number of unique tokens in all of the sequences $y_t^{(i)} \in \mathcal{X}$. Moving to decoder, it has layers usually used in transformer, depend on hyperparameters. Generator should generate sequence the most similar sequence to $\hat{y}_t^{(i)}$, or at the best $\hat{y}_t^{(i)}$. Simplified structure of the generator is pictured on Figure 1.

4.3 Discriminator

Discriminator as input get generated sequence by generator $\hat{y}_{t+1}^{(i)}$, sequence, what generator got as input $y_{t+1}^{(i)}$, and vector of properties $v^{(i)}$ from training set, the same the generator got. In training phase discriminator gets sequence from the training set $y_{t+1}^{(i)}$ with correct properties $v^{(i)}$, getting logits, and then we give generated sequence $\hat{y}_{t+1}^{(i)}$ and again properties $v^{(i)}$, getting another logits. On that set of logits, we count cost function, which R_1[13] regularization. R_1 regularization is counted on gradients given on by passing the true sequence $y_{t+1}^{(i)}$ throught the discriminator. In other words, while training the discriminator D we give sequences vector $v^{(i)}$ and sequence $y_{t+1}^{(i)}$ getting logits $o_{t+1}^{(i)} = D(v^{(i)}, y_{t+1}^{(i)}; \theta_{disc})$. Then to the discriminator we pass generated sequence $\hat{y}_{t+1}^{(i)}$ and vector $v^{(i)}$, getting $\hat{o}_{t+1}^{(i)} = D(v^{(i)}, \hat{y}_{t+1}^{(i)}; \theta_{disc})$. Getting logits $o_{t+1}^{(i)}$ and $\hat{o}_{t+1}^{(i)}$ we pass them to loss function, getting information to train both, generator (to trick discriminator) and discriminator (to better classify). We add to discriminator's loss

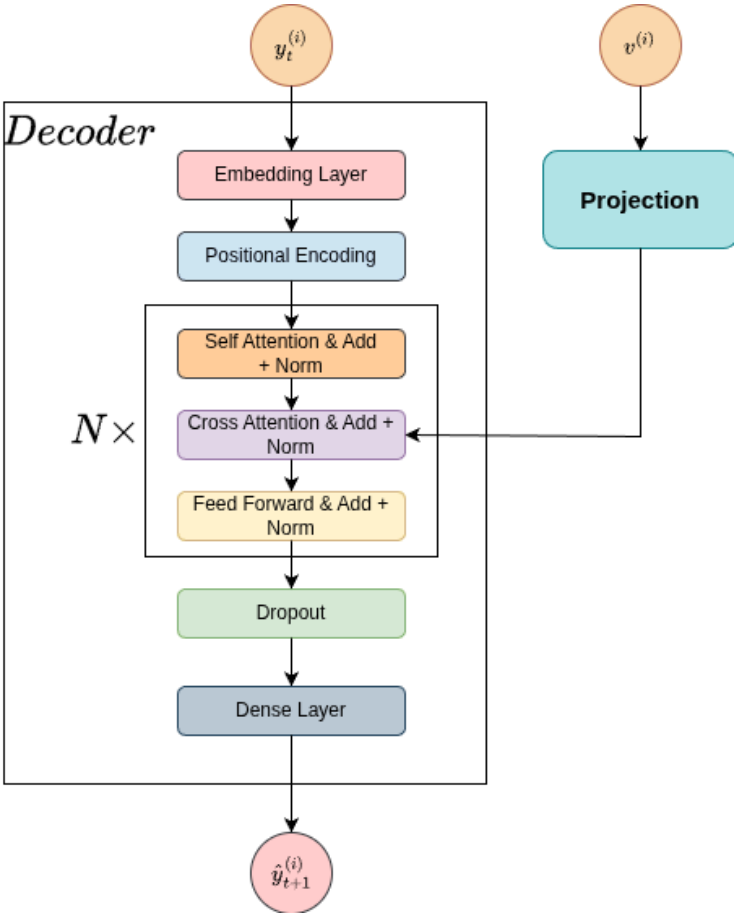


Figure 1: Simplified structure of the generator, where N is hyperparameter, that defines the number of attention blocks and fully connected network

function $R_1 = \frac{\gamma}{2}\mathbb{E}_{x\sim p_{data}}[||\nabla D_x(x)||^2]$, where $\nabla D_x(x)$ are gradients obtained from passing the true sequence $y_{t+1}^{(i)}$ through the discriminator D.

The discriminator in TGSGAN is an artifical neural network, which is inspired by model from "Convolutional Neural Networks for Sentence Classification"[7]. In the discriminator, an embedding layer is used to improve sequence pattern recognition. Next, layers add Gaussian noise, making the discriminator's task more difficult. Next, instead of traditional recurrent layers, convolutional layers are used, which are usually better suited for sequence processing. The outputs of convolutional layers are normalized using spectral normalization[14], which stabilizes training. The spectral normalization output is processed by the ReLU activation function with Gaussian noise. Next, max-pooling is added, and all of this is combined with layers, what process properties. Then we add some dropout. The simplified structure of the discriminator is pictured in Figure 2.

4.4 Discrete and continuous form of the molecules

Generative adversarial networks were originally trained for processing images. Training generative adversarial networks, however needs a different approach. We have four standard possibilities. Sequences generated by the generator in a discrete form $\hat{u}_{t+1}^{(i)} \in \mathbb{Z}^T$, in discrete one hot form $\hat{onehot}_{t+1}^{(i)} \in \mathbb{Z}^{T\times\mathfrak{v}}$, in simple logits form $\hat{o}_{t+1}^{(i)} \in \mathbb{R}^{T\times\mathfrak{v}}$ and simple probabilities $\hat{\sigma}_{t+1}^{(i)} \in \mathbb{Z}^{T\times\mathfrak{v}}$.

In the case $\hat{u}_{t+1}^{(i)}$ training would be almost imposible due to vanishing gradients. To transform data into form $\mathbb{Z}^T$ we should use operation argmax, which kills gradients. Model during the last phase of backpropagation, in which takes the gradients would not get valuable gradient, because the operation argmax is not differentiable; therefore the gradients would be nearly everywhere zero. In the case with $\hat{onehot}_{t+1}^{(i)}$ there is the same problem.

4.4.1 Logits and probabilities of sequences

Thirt and fourth approach in which sequences are directly from generator $\hat{o}_{t+1}^{(i)}$ and $\hat{\sigma}_{t+1}^{(i)}$ would have other kind of problem. Discriminator because it would get continuous variables, there would not be a problem with differentiability, but another, more specific problem. A generator in such a configuration would probably generate really "smoothed" logits or probabilities. In that case, the discriminator would have an easy task to distinguish which sequences are generated by the generator and which are from the training data. However, loss functions from GeometricGAN[10] address this problem; however, they are still not sufficient for good training. There is a method to ensure the differentiability of generated samples, and makes logits or probabilities more "sharp" and ensures good training, named the Gumbel-softmax method[15][12].

.

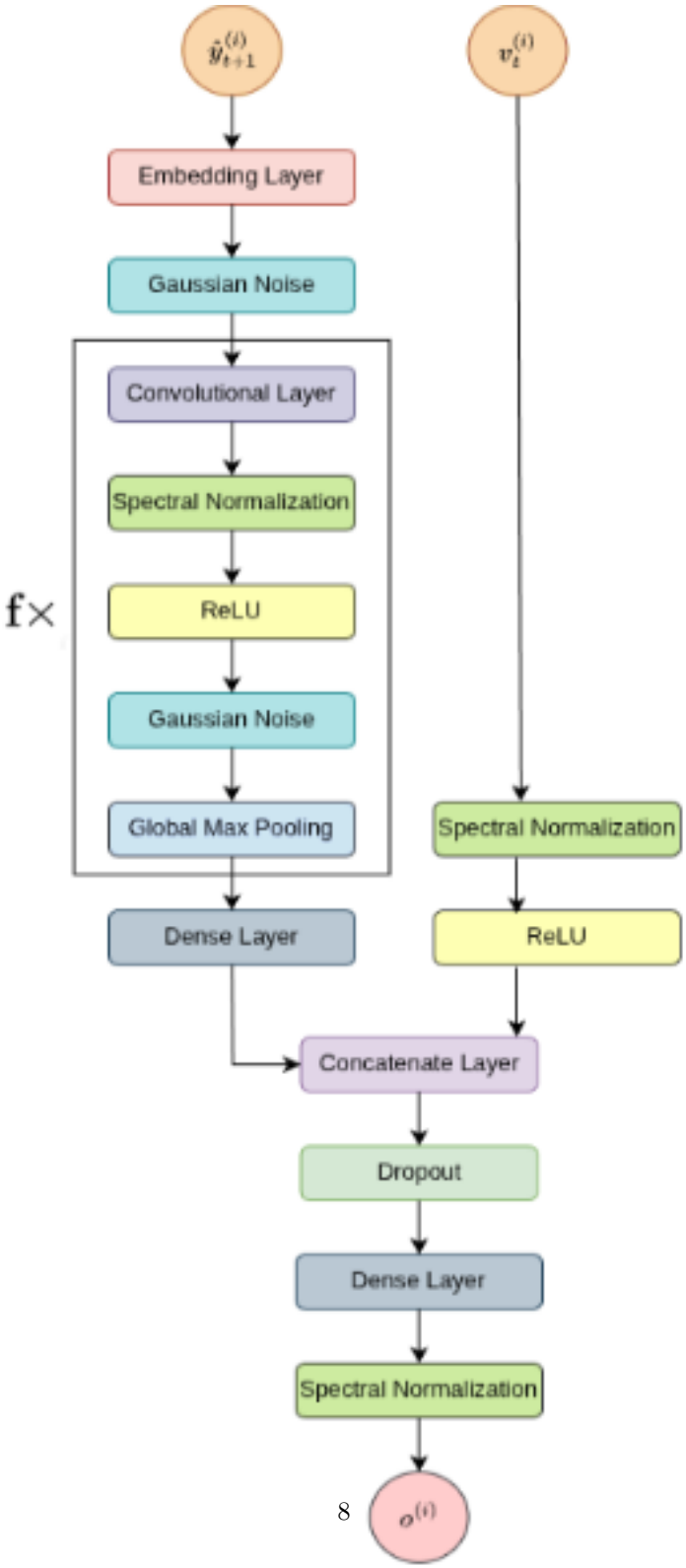


Figure 2: Simplified structure of discriminator, where f is size of filters for convolutional layers.

4.4.2 Gumbel-softmax method

Instead of using standard logit form $\hat{o}_{t+1}^{(i)}$ or probabilities $\hat{\sigma}_{t+1}^{(i)}$ as inputs to discriminator we give sequences in form:

$\hat{y}_{t+1}^{(i)} = \sigma(\beta(\hat{o}_{t+1}^{(i)} + g_t^{(i)}))$,

where $g_t^{(i)} = -log(-logU_t^{(i)})$, $U_t^{(i)} \sim Uniform(0,1)$, $\sigma(x_j) = \frac{e^{x_j}}{\sum_{k=1}^{n} e^{x_k}}$, and β it is hyperparameter. Gumbel-softmax method makes sequence $\hat{y}_{t+1}^{(i)}$ is still differentiable because does not use the operation argmax, and it makes sequence more one-hot-like, which makes the generator not only make more "sharp" probabilities but also generate better sequences.

Despite significant improvements in the generation sequence with the Gumbel-softmax method, the generator still has some problems with sequence processing, so it is useful to pretrain the model with more proven methods, such as standard maximum likelihood estimation. Therefore, TGSGAN was pretrained using standard maximum-likelihood estimation with teacher forcing.

4.5 Cost functions

The cost functions on which the model was trained are from Geometric GAN[10]. Those cost functions differ from the standard cost functions typically used in GAN training. The discriminator's cost function is based on classical SVC classification, in which we aim to find the widest "road" between the classes in the sample space. To the discriminator, we also add R_1[13] regularization with $\gamma = 20$, which stabilizes training. Discriminator's loss function looks like:

$$\mathrm{D}_{loss} = \mathbb{E}_{x\sim p_{data}}[\max(0, 1-D(x))]+\mathbb{E}_{z\sim p_z}[\max(0, 1+D(G(z)))]+\frac{\gamma}{2}\mathbb{E}_{x\sim p_{data}}[||\nabla D(x)||^2], \quad (2)$$

Meanwhile, the generator's loss function is:

$$G_{loss} = -\mathbb{E}_{z\sim p_z}[D(G(z))]. \quad (2)$$

4.6 Hyperparameters

4.6.1 Learning rates and optimizers

The model was trained using the RMSProp alorithm for both the generator and the discriminator. To train well-trained GAN, the learning rate and its schedules are crucial. Initial generator's learning rate was set as $1 \cdot 10^{-5}$ and the discrminator's initial learning rate was set as $1 \cdot 10^{-3}$. As schedulers, we choose cosine decay scheduler[11].

4.6.2 Epochs

The model was trained for 10 epochs, and training took approximately 11 hours on a single RTX 4070 Ti Super.

4.6.3 Batch size

The training set was divided into 32 batches.

4.6.4 Inverse temperature

The greater the inverse temperature β in the equation $\hat{y}_{t+1}^{(i)} = \sigma(\beta(\hat{o}_{t+1}^{(i)} + g_t^{(i)}))$, then samples are more likely to be one hot encoded, and the smaller β, the smoorher the probabilities are. A higher β at the beginning of training is not optimal; therefore it is better to set the initial inverse temperature to a small value and progressively increase it. The inverse temperature scheduler looks like: $\beta = \beta_{final}^{(\frac{epoch}{nepochs})}$, where β_{final} is the maximum inverse temperature, what model can be trained. The final inverse temperature β_{final} was set to 10.

5 Evaluation

The quality of the model can be assessed in several different ways. We can take property vectors from the test set, generate sequences of molecules based on those properties, and measure the error between the test-set properties and the generated-sequence properties. We can also check how many test-set samples are already in the training set and assess the diversity of the generated samples. Model, what generates the same sequences over and over is not what we usually need in drug discovery. Also, we can check not only the diversity of molecular sequence sets across the entire test set, but also within a single randomly chosen vector $v^{(k)}$.

To evaluate generative adversarial network it is also trained transformer without GAN structure with classical teacher forcing and maximum likelihood estimation. Such that transformer was trained with 32 batch size, constant learning rate $1 \cdot 10^{-3}$ with Adam algorithm [8], 10 epochs with the same transformer hyperparameters as generator, and cost function as masked categorical crossentropy. Probabilities for generative adversarial networks were chosen with the argmax operation, while transformers were chosen by both the argmax operation and with a method to sample more randomly with temperature 1.2 (transformer rand. in evaluation). However, the main motivation for choosing generative adversarial networks was to obtain more "interesting" samples, which is not easy to compare with transformers; therefore, the above evaluation methods are only suggestions of how GANs perform. Some of the generated samples are displayed in appendix A.

Model	MSE metric
GAN	33.410
Transformer	16.913
Transformer Rand.	34.255

Table 1: Mean squared error for the test set.

5.1 Model's error

The model was trained on the qm9 database [16], which contains 130,831 distinct samples. The database was divided into training, validation and test set. The database was divided into training, validation, and test sets. The training and validation sets comprise 80% of the database, and the test set comprises 20%. Database qm9 has sequences of molecules SMILES, which contain molecules with carbon atoms, oxygen atoms, nitrogen atoms, and fluorine atoms, which are converted to SELFIES with SELFIES Library, and some properties of those sequences. Due to difficulties in evaluating the error for some properties that require physical synthesis, the error was counted only for properties that could be easily obtained by algorithms without synthesis and were previously added to the dataset. Those properties are: quantitative estimation of druglikeness, Log P[18], tpsa (polar surface area), and mean molecular weight. The error metric was the mean squared error. MSE scores are included in Table 1.

5.2 Repeatability of molecular sequences

5.2.1 Generated samples not in training set

In drug discovery with generative models, we usually want to generate samples not included in the training set, but new, unseen molecules. The number of generated samples not in the training set is included in Table 2.

Model	Samples not in training set
GAN	21211
Transformer	19035
Transformer Rand.	22207

Table 2: Samples not included in the training set out of 83731 samples in the training set.

5.2.2 Repeatability of molecular sequences for the test set properties

We can also check how many different molecules are generated for the entire test set. The number of unique molecules across all test sets is shown in Table 3.

Model	Number of unique samples
GAN	21690
Transformer	20055
Transformer Rand.	25296

Table 3: Number of unique sequences for all the properties from the test set with 26167 samples.

5.2.3 For one vector of properties

The number of unique sequences for a single vector $v^{(k)}$ chosen randomly is included in Table 4.

Model	Number of unique sequences
GAN	501
Transformer	1
Transformer Rand.	779

Table 4: Number of unique sequences for single vector $v^{(k)}$ chosen randomly, where the number of generated samples is 1000.

6 Conclusions

Generative adversarial networks and transformers, despite most generated samples already being in the training set, correctly learned the structures of molecular sequences. Despite lower error MSE for TGSGAN, the model that performs better must be chosen individually, due to the models' scores being really different on various tasks. For further improvements, the model can be trained

on datasets with even more samples and a wider range of atoms. Some of the generated samples are displayed in appendix A.

References

[1] Nicola De Cao and Thomas Kipf. Molgan: An implicit generative model for small molecular graphs, 2022.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[3] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

[4] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax, 2017.

[5] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation, 2019.

[6] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. 596(7873):583–589.

[7] Yoon Kim. Convolutional neural networks for sentence classification, 2014.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

[9] Mario Krenn, Florian Häse, AkshatKumar Nigam, Pascal Friederich, and Alan Aspuru-Guzik. Self-referencing embedded strings (selfies): A 100 *Machine Learning: Science and Technology*, 1(4):045024, October 2020.

[10] Jae Hyun Lim and Jong Chul Ye. Geometric gan, 2017.

[11] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017.

[12] Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling, 2015.

[13] Lars Mescheder, Andreas Geiger, and Sebastian Nowozin. Which training methods for gans do actually converge?, 2018.

[14] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks, 2018.

[15] Weili Nie, Nina Narodytska, and Ankit Patel. RelGAN: Relational generative adversarial networks for text generation. In *International Conference on Learning Representations*, 2019.

[16] Raghunathan Ramakrishnan, Pavlo O Dral, Matthias Rupp, and O Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1, 2014.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[18] Scott Wildman and Gordon Crippen. Prediction of physicochemical parameters by atomic contributions. *Journal of Chemical Information and Computer Sciences*, 39:868–873, 09 1999.

A Some generated molecules

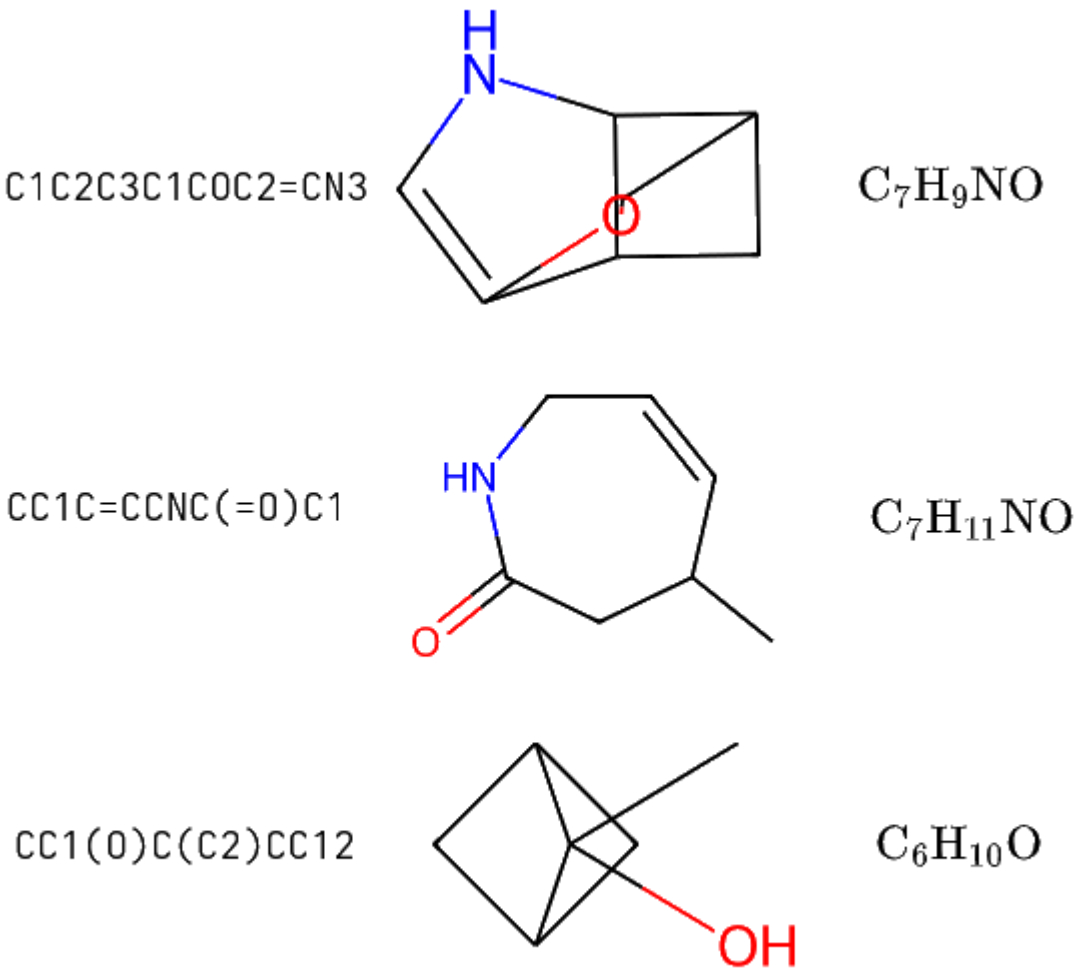


Figure 3: Molecules generated by TGSGAN, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

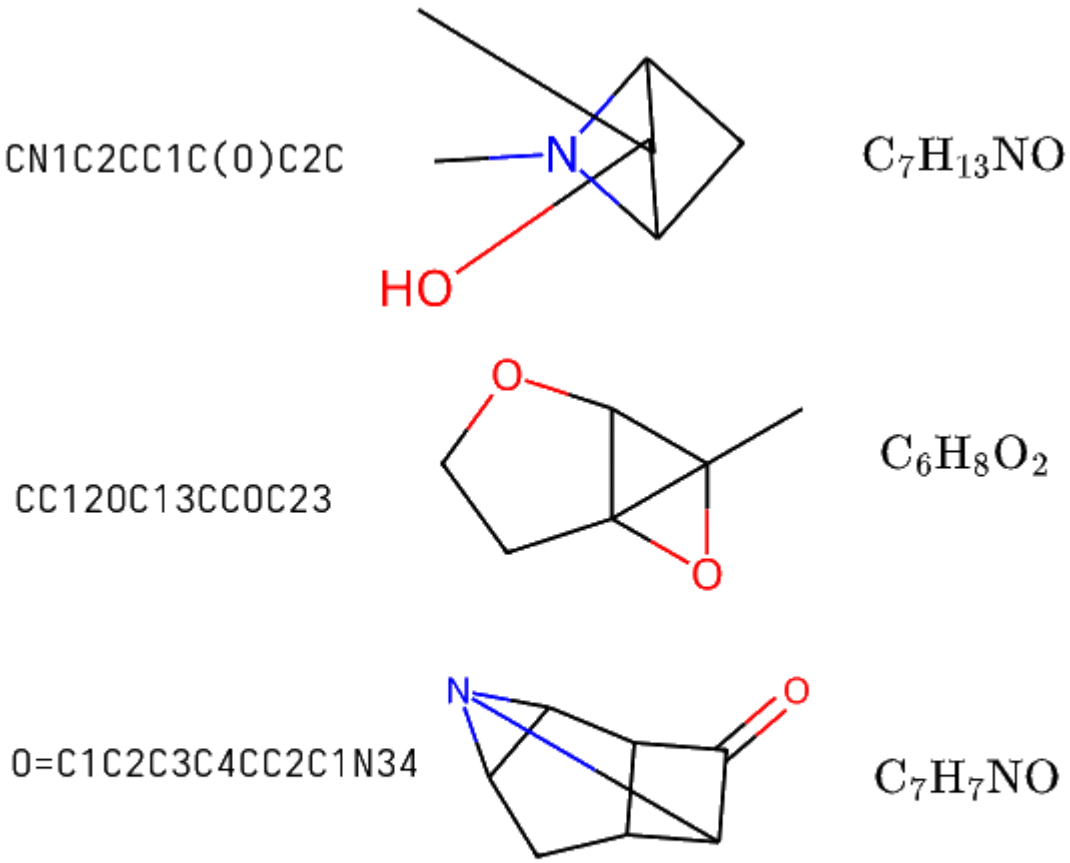
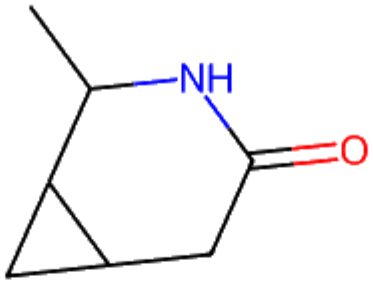


Figure 4: Molecules generated by TGSGAN, where on left there is SMILES strings, on middle there is 2D structure, and chemical formula on right.

CC1NC(=O)CC2CC12

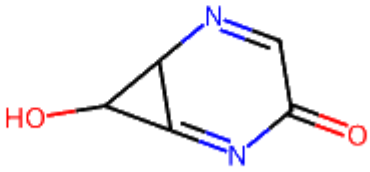


$C_7H_{11}NO$

O=COCCC=O

$C_4H_6O_3$

OC1C2=NC(=O)C=NC12



$C_5H_4N_2O_2$

Figure 5: Molecules generated by TGSGAN, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

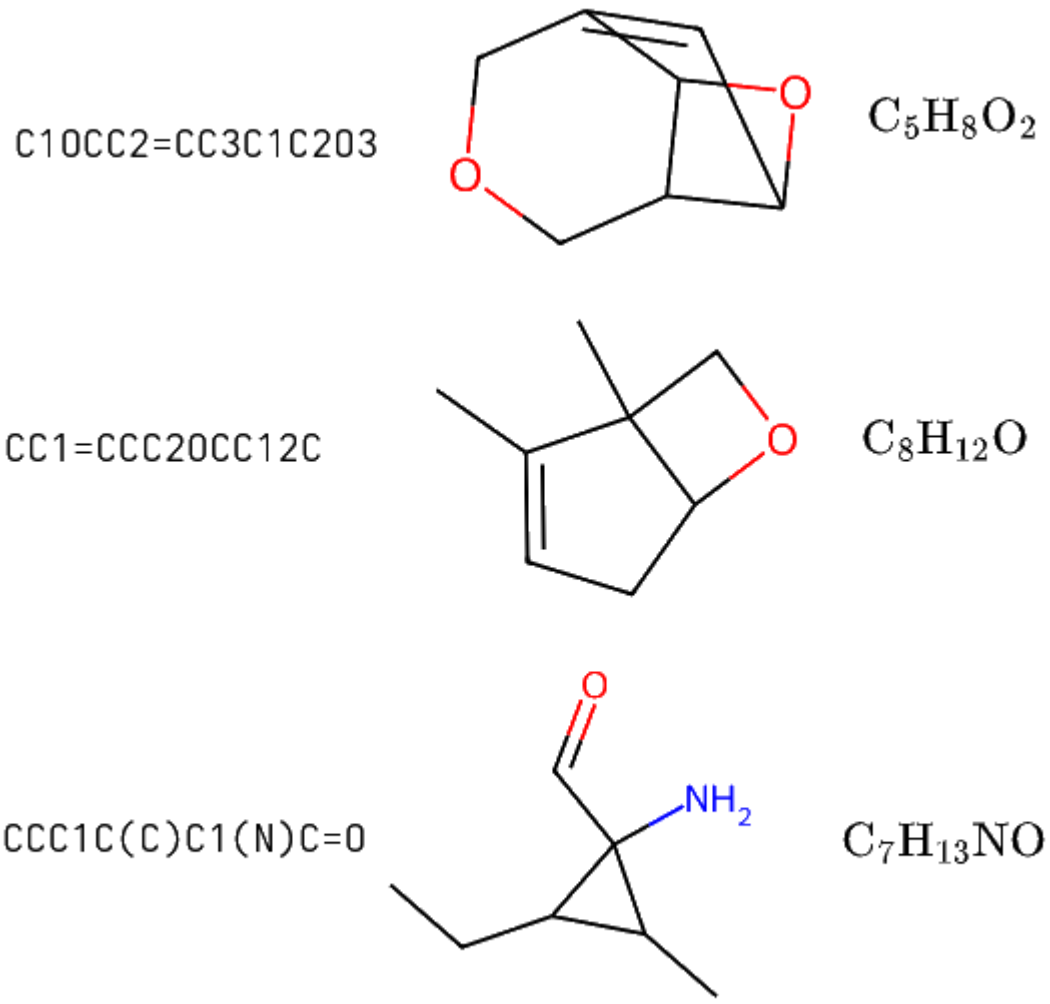


Figure 6: Molecules generated by transformer with argmax operation, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

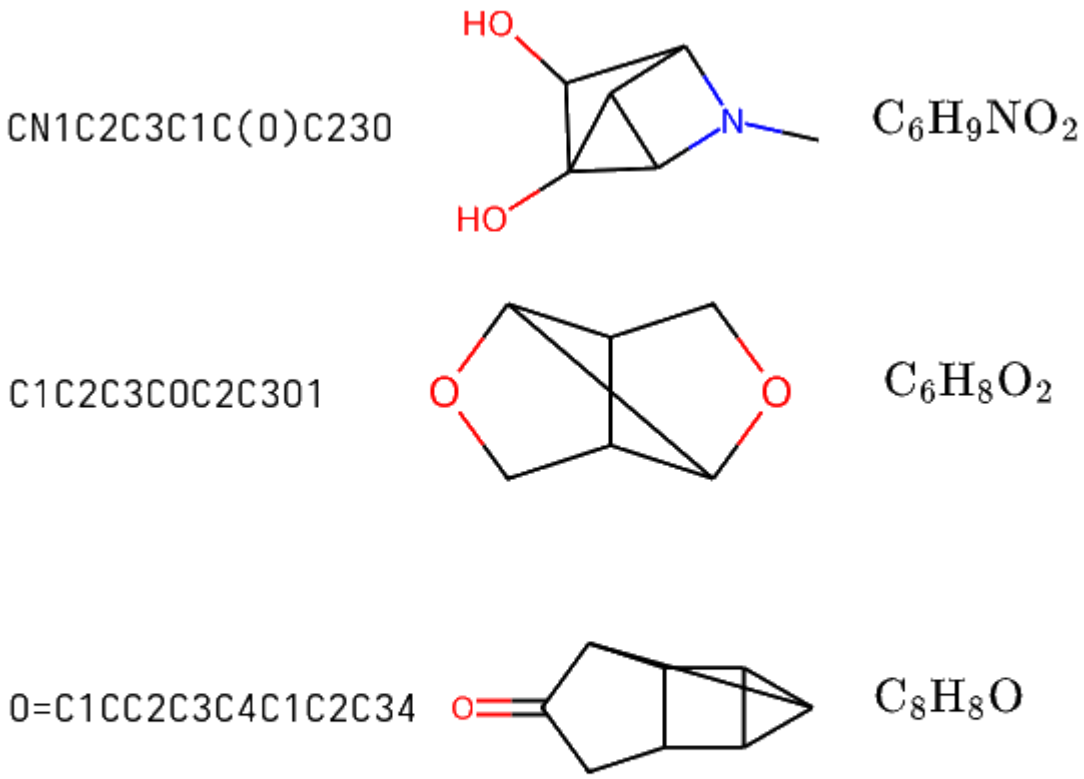


Figure 7: Molecules generated by transformer with argmax operation, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

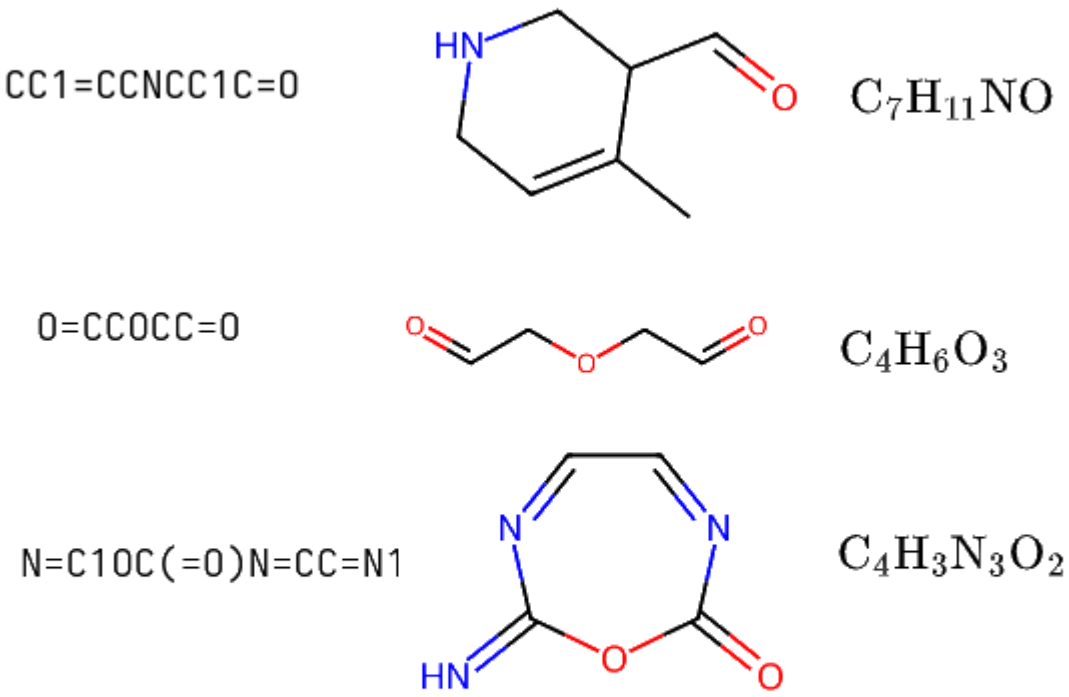


Figure 8: Molecules generated by transformer with argmax operation, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

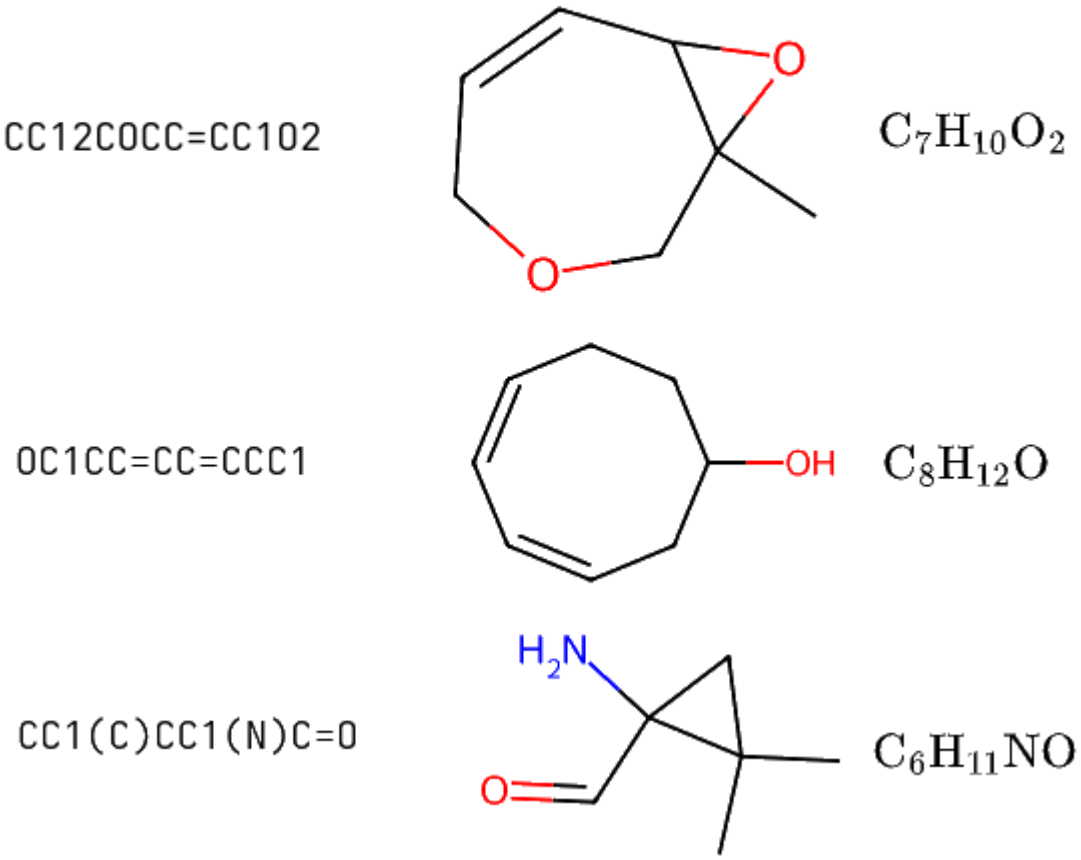


Figure 9: Molecules generated by transformer with method of more randomness of samples, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

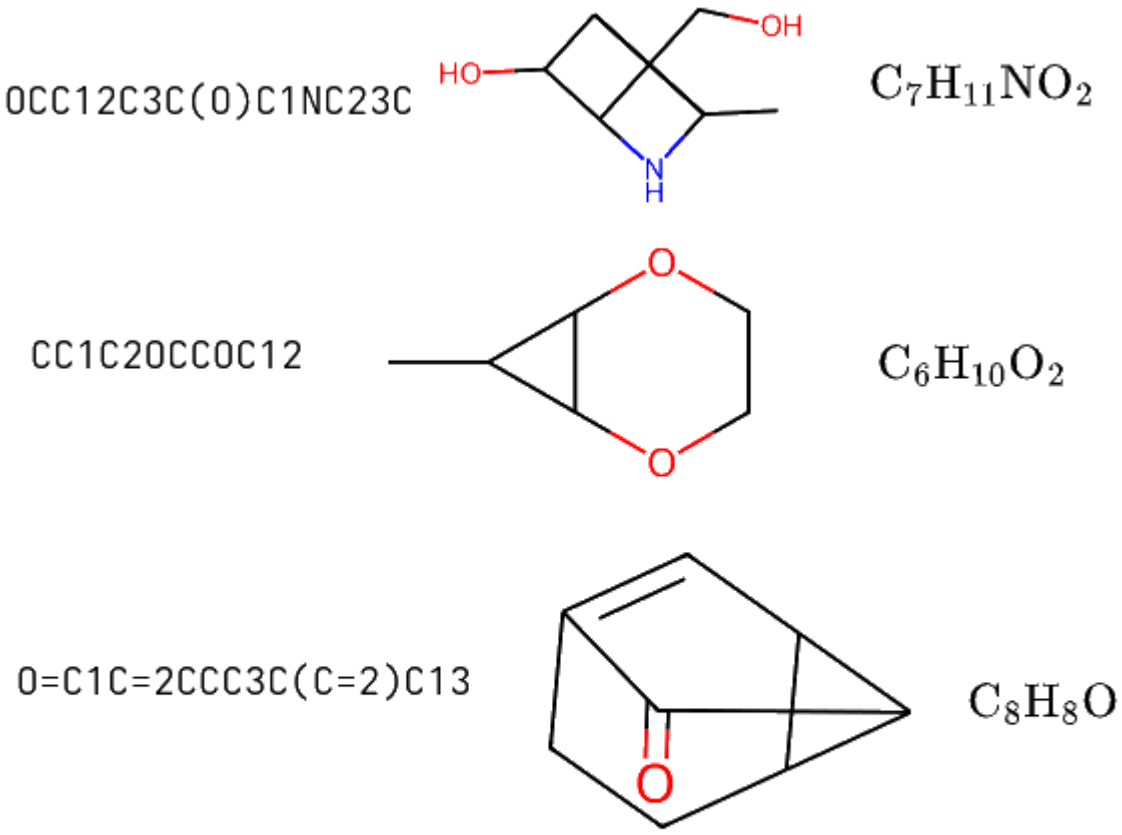


Figure 10: Molecules generated by transformer with method of more randomness of samples, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.

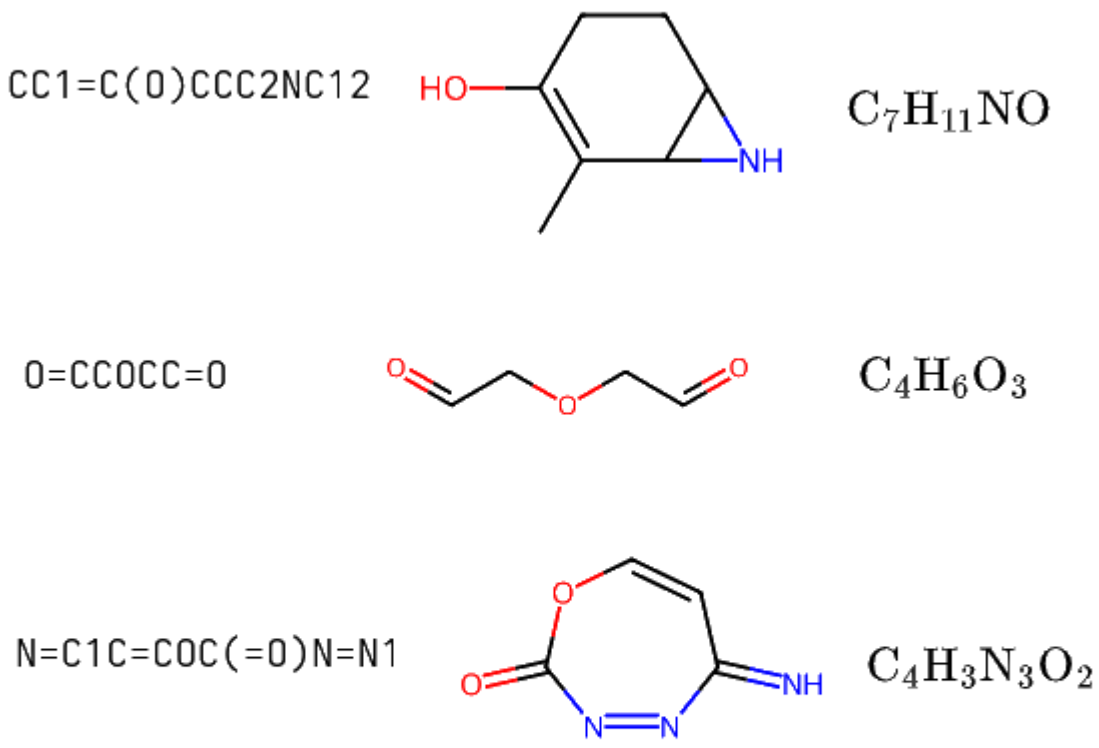


Figure 11: Molecules generated by transformer with method of more randomness of samples, where on left there are SMILES strings, on middle there is 2D structure, and chemical formula on right.